

6.1 Anatomía y Tipos de Colecciones en Dart

Todas las colecciones principales en Dart forman parte de la librería núcleo (`dart:core`) y aprovechan el sistema de genéricos (`<T>`) para garantizar que la recolección y manipulación de elementos sea *type-safe*.

Listas (`List<T>`)

Son colecciones ordenadas con acceso indexado basado en cero. Pueden ser de tamaño variable o de tamaño fijo.

- **Literales:** `List<String> nombres = ['Ana', 'Carlos'];`
- **Acceso e indexación:** $O(1)$ para lectura por índice (`nombres[0]`).

Conjuntos (`Set<T>`)

Colecciones no ordenadas de elementos **únicos**. Si intentas insertar un valor duplicado, el `Set` lo ignora automáticamente. Son internamente implementados con tablas hash, lo que brinda búsquedas ultrarrápidas $O(1)$.

- **Literales:** `Set<String> roles = {'admin', 'editor', 'user'};`
- **Utilidad:** Eliminar duplicados instantáneamente (`lista.toSet()`) o calcular intersecciones y uniones.

Mapas (`Map<K, V>`)

Estructuras clave-valor no duplicadas por clave. Son la representación directa de un objeto JSON.

- **Literales:** `Map<String, dynamic> usuario = {'id': 1, 'nombre': 'Ozzy'};`
- **Búsqueda:** Lectura rápida $O(1)$ dada una llave (`usuario['nombre']`).

La Interfaz `Iterable<T>`

`Iterable` es la clase abstracta padre de `List` y `Set`. Representa una secuencia de elementos que pueden ser recorridos secuencialmente.

Diferencia Crítica: Las operaciones sobre un `Iterable` son **Lazy** (perezosas/diferidas). Los filtros como `.where()` o transformaciones como `.map()` no procesan los datos en memoria hasta que el iterador es transformado explícitamente a una lista (`.toList()`) o consumido en un bucle `for-in`.

6.2 Operadores Coleccionales Avanzados: Spread y Control Flow

Dart posee características sintácticas de primera clase para manipular y combinar colecciones directamente dentro de sus inicializadores literales:

Spread Operator (`...`) y Null-Aware Spread (`...?`)

Permite desempaquetar o expandir los elementos de una colección existente dentro de otra.

```
List<int> bloque1 = [1, 2];
List<int> bloque2 = [3, 4];
List<int>? bloqueOpcional;

List<int> consolidado = [...bloque1, ...bloque2, ...?bloqueOpcional]; // [1, 2, 3, 4]
```

Collection if y Collection for

Permite agregar elementos condicionalmente o proyectar listas completas directamente dentro de la definición de una nueva colección:

```
bool esAdmin = true;
List<String> listaNavegacion = [
  'Inicio',
  'Perfil',
  if (esAdmin) 'Panel de Control', // Collection IF
  for (var i = 1; i <= 3; i++) 'Opción Extra $i', // Collection FOR
];
```

6.3 Métodos de Orden Superior y Transformaciones (map, where, fold, reduce)

Los métodos de orden superior transforman datos de forma declarativa e inmutable:

- `.map((e) => ...)`: Transforma cada elemento de tipo `T` a tipo `R`. Retorna un `Iterable<R>`.
- `.where((e) => bool)`: Filtra los elementos que cumplen la condición booleana.
- `.fold(initialValue, (acc, e) => ...)`: Reduce la colección a un único valor acumulado partiendo de un valor inicial.
- `.reduce((acc, e) => ...)`: Similar a `fold`, pero toma el primer elemento de la colección como valor inicial (falla si la lista está vacía).

6.4 Ejemplo

Creen el archivo `bin/d6_colecciones.dart` en VS Code. El siguiente programa simula la recepción de una respuesta de una API REST con usuarios y materias, demostrando filtrado, mapeo inmutable y agregación.

```
/// Módulo D6: Colecciones e Iterables en Dart
/// Demostración de List, Set, Map, Spread Operators y Métodos de Orden Superior.

void main() {
  print('=====');
```

```

print(' PROCESADOR DE COLECCIONES Y RESPUESTAS JSON - ESCOM ');
print('=====\\n');

// 1. Simulación de un Payload de API REST (List<Map<String, dynamic>>)
final List<Map<String, dynamic>> respuestaApi = [
  {
    'id': 'USR-101',
    'nombre': 'Carlos Ruiz',
    'calificaciones': [8, 9, 7, 10],
    'materiasInscritas': ['DAMN', 'Redes', 'DAMN'], // Duplicado intencional
    'activo': true,
  },
  {
    'id': 'USR-102',
    'nombre': 'María López',
    'calificaciones': [6, 5, 7],
    'materiasInscritas': ['BD', 'Sistemas Operativos'],
    'activo': false,
  },
  {
    'id': 'USR-103',
    'nombre': 'Jorge Ramos',
    'calificaciones': [10, 10, 9, 10],
    'materiasInscritas': ['DAMN', 'Ciberseguridad', 'IA'],
    'activo': true,
  },
];

// 2. Filtrado con .where() y transformación con .map()
print('--- 1. Alumnos Activos con Promedio >= 8.0 ---');
final Iterable<Map<String, dynamic>> alumnosActivos = respuestaApi.where((usr)
=> usr['activo'] == true);

// Mapeo declarativo a un modelo simplificado
final List<String> cuadroHonor = alumnosActivos.where((usr) {
  final List<int> notas = List<int>.from(usr['calificaciones']);
  final double promedio = notas.fold(0, (sum, n) => sum + n) / notas.length;
  return promedio >= 8.0;
}).map((usr) {
  return 'Alumno: ${usr['nombre']} (${usr['id']})';
}).toList();

cuadroHonor.forEach((linea) => print(' - $linea'));

// 3. Extracción de elementos únicos usando Set y Spread Operator
print('\\n--- 2. Catálogo Unificado de Materias Impartidas (Set) ---');
final Set<String> catalogoMaterias = {};

for (final usr in respuestaApi) {
  final List<String> materias = List<String>.from(usr['materiasInscritas']);
  catalogoMaterias.addAll(materias); // Set elimina automáticamente 'DAMN'
  duplicado
}

```

```
print('Materias únicas registradas: $catalogoMaterias');

// 4. Collection IF y Spread Operator (...) en la construcción de menús de UI
print('\n--- 3. Construcción Dinámica de Colección para UI ---');
final bool incluirInactivos = false;

final List<String> listaNombresFinal = [
  '--- LISTA GENERAL DE USUARIOS ---',
  ...cuadroHonor, // Spread de la lista generada previamente
  if (incluirInactivos) 'María López (INACTIVA)', // Collection IF
  for (final materia in catalogoMaterias) 'Sección Materia: $materia', //
Collection FOR
];

listaNombresFinal.forEach((item) => print(' $item'));

print('\n=====');
}
```

Instrucciones de Ejecución en VS Code:

```
dart run bin/d6_colecciones.dart
```

6.5 Ejercicios Prácticos

Implementen los siguientes dos programas en su entorno local para dominar las colecciones y métodos de orden superior.

Ejercicio 1: Eliminación de Duplicados y Deducción Set

Dado un registro de accesos por boleta de alumnos ingresando a la biblioteca (`List<String>` con múltiples boletas repetidas):

1. Convierte la lista a un `Set<String>` para obtener únicamente las boletas únicas.
2. Utiliza el método `.where()` para filtrar aquellas boletas que inicien con el año '2021' (tip: usa `.startsWith()`).
3. Retorna e imprime una nueva lista ordenada alfabéticamente utilizando `.toList()..sort()`.

Ejercicio 2: Pipeline de Agregación de Ventas de Cafetería

Dada una lista de pedidos representados como mapas (`List<Map<String, dynamic>>` donde cada mapa incluye 'producto', 'precio' (`double`) y 'categoria'):

1. Filtra los pedidos pertenecientes únicamente a la categoría 'Bebidas' usando `.where()`.
2. Mapea la lista filtrada para obtener solo los precios.

3. Calcula el total de ingresos por bebidas utilizando `.fold()`.
 4. Muestra en consola el catálogo filtrado y el monto total formateado a dos decimales.
-